

Health Transformation Review — Developer Guide

Version: 2026-03-10 | Stack: Next.js 16 · React 19 · Sanity CMS · Supabase · FastAPI · LlamaIndex · Gemini · Tailwind v4

Table of Contents

1. Tech Stack

2. Repository Layout

3. Environment Variables

4. Architecture & Data Flow

5. Getting Started (Local Dev)

6. Routing & Page Structure

7. Component Architecture

8. Color System

9. Sanity CMS — All Schemas

10. Static Data Layer

11. RAG Pipeline (AI Analyst)

12. Dashboard System

13. Auth & Middleware

14. Scripts Reference

15. API Routes

16. How to Add a New Pillar

17. How to Add a New State

18. Deployment

19. Known Issues & Roadmap

1. Tech Stack

Frontend (Next.js)

Layer	Technology	Version	Notes
Framework	Next.js	16.1.6	App Router (not Pages Router)
UI Library	React	19.0.0-rc	Pre-release — stable for production
Styling	Tailwind CSS	4.0.0-beta.9	v4 beta — uses <code>@theme</code> in CSS, not <code>tailwind.config.js</code>
Styled Components	styled-components	6.3.8	Used for a few legacy components only
CMS	Sanity	latest	Studio embedded at <code>/studio</code> route
CMS Client	next-sanity	latest	Includes <code>createClient</code> + live preview utilities
Database / Auth	Supabase	2.89.0	Postgres + pgvector + auth
Supabase SSR	@supabase/ssr	0.9.0	Server-side session handling in middleware
Maps	react-simple-maps	1.0.0	D3-backed SVG US map
Type Safety	TypeScript	5.x	Strict mode enabled
Language	Node.js	20.x	Dev and build environment

Backend AI Brain (Python)

Layer	Technology	Version	Notes
API Framework	FastAPI	≥0.111	Async HTTP server; CORS configured for frontend
ASGI Server	uvicorn	≥0.30	Runs FastAPI; <code>---reload</code> flag for dev

Layer	Technology	Version	Notes
AI Orchestration	LlamaIndex Core	≥0.11	VectorStoreIndex, ChatMemoryBuffer, streaming
LLM	Google Gemini Flash Lite	<code>gemini-flash-lite-latest</code>	Via <code>llama-index-llms-google-genai</code>
Embedding Model	Google text-embedding-004	—	768-dim; via <code>llama-index-embeddings-google-genai</code>
PDF Ingestion	LlamaIndex SimpleDirectoryReader	—	Auto-discovers all <code>.pdf</code> files in <code>backend/data/</code>
HTTP Client	httpx	≥0.27	Async Sanity GROQ HTTP API calls
Vector Store	LlamaIndex in-memory + disk	—	Persisted to <code>backend/storage/</code> (gitignored)

Important version notes:

- Next.js reports as `^16.1.6` in `package.json` — treat as Next.js 15 app router series.
- Tailwind v4 is a **breaking change** from v3: no `tailwind.config.js` theme extensions for custom colors — all color tokens live in `globals.css` via `@theme`.
- React 19 RC ships with concurrent features; no `ReactDOM.render()` — use `createRoot()`.
- The Python backend owns all AI logic. The Next.js `/api/chat` route is a **thin proxy** — it forwards requests to the Python backend and streams responses back. No AI logic lives in TypeScript.

2. Repository Layout

```

Vermont-Health-Platform/
├── backend/
│   ├── main.py
│   ├── requirements.txt
│   ├── .env
│   └── .env.example
├── data/
│   ├── wyman-report.pdf
│   ├── *.pdf
│   └── storage/
├── generated/
│   ├── docstore.json
│   └── ...
├── frontend/
│   ├── app/
│   │   ├── layout.tsx
│   │   ├── globals.css
│   │   ├── page.tsx
│   │   └── api/
│   │       ├── chat/route.ts
│   │       ├── digest/route.ts
│   │       ├── rht-states/route.ts
│   │       ├── search/route.ts
│   │       └── subscribe/route.ts
│   ├── dashboard/
│   │   ├── layout.tsx
│   │   ├── page.tsx
│   │   ├── DashboardIndexClient.tsx
│   │   └── [state]/
│   │       ├── page.tsx
│   │       └── StateDetailClientPage.tsx
│   ├── policy/
│   │   ├── page.tsx
│   │   └── [slug]/page.tsx
│   ├── economics/
│   ├── technology/
│   ├── clinical/
│   └── equity/

```

- Python AI Brain (FastAPI + LlamaIndex)
- FastAPI server (all AI/RAG logic lives here)
- Python dependencies
- Backend secrets (gitignored – copy from .env.example)
- Template: GOOGLE_API_KEY, Sanity vars, FRONTEND_URL
- Drop PDF files here for ingestion
- Ingested at startup
- Add more PDFs here, then POST /api/ingest
- Persisted VectorStoreIndex (gitignored, auto-generated)
- Presence of this file = fast startup
- LlamaIndex index files
- Next.js application (primary codebase)
- App Router pages and API routes
- Root layout (Header + AppShell + Footer)
- SINGLE SOURCE OF TRUTH for all colors (@theme)
- Home page
- Thin proxy → Python backend /api/chat
- Email digest trigger
- RHT state data API
- Full-text search endpoint
- Email subscription endpoint
- State Performance Dashboard
- Dashboard layout with sidebar
- Dashboard index (map view)
- Client component for map
- Dynamic state detail pages
- Server component (fetches data)
- Client (tabs, charts)
- Policy pillar hub + articles
- PillarHub template
- Individual article
- Economics pillar hub + articles
- Technology pillar hub + articles
- Clinical pillar hub + articles
- Equity pillar hub + articles

└─ academy/	← Academy learning platform
└─┬─ page.tsx	← Academy home
└─┬─ courses/[slug]/	← Course detail
└─┬─ modules/[slug]/	← Module detail
└─┬─ webinars/[slug]/	← Webinar detail
└─└─ glossary/	← Glossary with pillar filter
└─ account/	← Protected user account (requires auth)
└─ login/	← Auth login page
└─ signup/	← Auth signup page
└─ reset-password/	← Password reset flow
└─ search/	← Full-text search results
└─ chat/	← AI Analyst standalone chat page
└─ advisory/	← Advisory hub
└─ hti-dashboard/	← HTI (Health Transformation Index) dashboard
└─ trending-topics/	← Trending topics page
└─ privacy/	← Privacy policy stub
└─ terms/	← Terms of service stub
└─ sitemap/	← Sitemap stub
└─ components/	← Shared React components
└─┬─ Header.tsx	← Top navigation bar
└─┬─ Footer.tsx	← Site footer
└─┬─ AppShell.tsx	← Left/right sidebar shell (auto-collapses)
└─┬─ HomeSidebar.tsx	← Left sidebar (pillar nav links)
└─┬─ RightSidebar.tsx	← Right sidebar (AI chat suggestions)
└─┬─ CommandPalette.tsx	← Cmd+K command palette
└─┬─ VideoLibrary.tsx	← Multi-pillar video library
└─┬─ TickerContext.tsx	← Context provider for news ticker
└─┬─ TooltipContext.tsx	← Tooltip context provider
└─┬─ dashboard/	← Dashboard-specific components
└─└─ RHTScorecard.tsx	← RHT program scorecard
└─ templates/	← Full-page template components
└─┬─ PillarHub.tsx	← Template for all 5 pillar hub pages
└─┬─ ArticleEngine.tsx	← Template for individual articles
└─┬─ AcademyModuleEngine.tsx	← Academy module renderer
└─┬─ AcademyModuleLayout.tsx	← Academy module layout wrapper
└─└─ HubPageTemplate.tsx	← Generic hub page template
└─ lib/	← Utilities, data, and service clients
└─┬─ sanity.ts	← Sanity client + urlFor() helper
└─┬─ supabase.ts	← Supabase browser client
└─┬─ rag.ts	← RAG helpers (embed, search, buildContext)
└─┬─ chat.ts	← Chat client-side utilities
└─┬─ ticker.ts	← Ticker/news data fetcher
└─┬─ utils.ts	← General utility functions
└─┬─ dashboard-types.ts	← TypeScript types for dashboard
└─┬─ sanity-dashboard-queries.ts	← GROQ queries for dashboard
└─┬─ context/	
└─└─┬─ DashboardContext.tsx	← React context for dashboard state
└─└─└─ ProgramContext.tsx	← React context for RHT program state
└─┬─ data/	← Static data (pre-Sanity / seed reference)
└─└─┬─ performance-index-data.ts	← 50-state performance metrics
└─└─┬─ rht-program.ts	← RHT state profiles + RHTProfile type
└─└─┬─ hospital-data.ts	← Hospital reference data
└─└─┬─ rht-awards.ts	← RHT award amounts by state
└─└─└─ states.ts	← State metadata (names, abbreviations)
└─┬─ hooks/	
└─└─ useSolvencySimulation.ts	← Hospital solvency simulation hook
└─ sanity/	← Sanity Studio configuration
└─┬─ schemaTypes/	← All 21 document schemas (see §9)
└─┬─ generate_sanity_content.py	← AI content generator (Gemini)
└─└─ ...	← Studio desk config, plugins
└─ scripts/	← One-off seed and sync scripts (Node/tsx)
└─┬─ seed-content.ts	← Seed initial Sanity content
└─┬─ seed-hospitals.ts	← Seed hospital documents
└─┬─ seed-sanity-performance-index.ts	← Seed state performance index
└─┬─ seed-sanity-rht.ts	← Seed RHT state profiles
└─└─ sync-embeddings.ts	← Legacy: Sanity → Supabase pgvector (superseded by
Python backend)	
└─┬─ middleware.ts	← Supabase auth guard for /account routes
└─┬─ next.config.ts	← Next.js config (styled-components compiler)
└─└─ .env.local	← Environment variables (never commit)
└─ supabase/	
└─└─ setup-rag.sql	← One-time pgvector setup SQL (for Supabase auth; RAG

now in Python)

└ USER_MANUAL.md

└ DEVELOPER_GUIDE.md

← End-user documentation

← This file

3. Environment Variables

There are **two separate .env** files — one for the Next.js frontend, one for the Python backend.

frontend/.env.local — Next.js app

Never commit this file.

```
# ——— Sanity CMS ———
# Project ID and dataset from sanity.io/manage
NEXT_PUBLIC_SANITY_PROJECT_ID=your-project-id
NEXT_PUBLIC_SANITY_DATASET=production
NEXT_PUBLIC_SANITY_API_VERSION=2023-10-01

# Read/write API token for server-side mutations and seeding scripts.
# Generate at: sanity.io/manage → your project → API → Tokens
# Required permissions: Editor (or higher for mutations)
SANITY_API_TOKEN=your-sanity-api-token

# ——— Supabase ———
# Found in: Supabase dashboard → Project Settings → API
NEXT_PUBLIC_SUPABASE_URL=https://your-project-ref.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-anon-key

# Service role key – server-side only, bypasses Row Level Security.
# Used by sync-embeddings.ts (legacy RAG sync) to write to content_embeddings.
# NEVER expose to the browser.
SUPABASE_SERVICE_ROLE_KEY=your-service-role-key

# ——— Google AI (Embeddings – legacy sync script only) ———
# Only needed if running sync-embeddings.ts (the legacy Supabase RAG sync).
# The AI Analyst chat itself no longer needs this key in the frontend –
# it proxies all AI calls to the Python backend.
GEMINI_API_KEY=your-gemini-api-key

# ——— Python AI Backend ———
# URL of the running Python FastAPI backend.
# Default: http://localhost:8000 (used if this var is absent).
# Set to your production backend URL for deployment.
PYTHON_BACKEND_URL=http://localhost:8000
```

backend/.env — Python AI Brain

Copy from **backend/.env.example** and fill in your values. Never commit this file.

```
# ——— Google AI ———
# Powers Gemini LLM (gemini-flash-lite-latest) + embeddings (text-embedding-004)
# Get from: https://aistudio.google.com/app/apikey
GOOGLE_API_KEY=your-google-api-key

# ——— Sanity CMS ———
# The Python backend fetches CMS content directly via GROQ HTTP API.
# Use a read-only API token (no write permissions needed).
SANITY_PROJECT_ID=your-project-id
SANITY_DATASET=production
SANITY_API_TOKEN=your-read-only-sanity-token
SANITY_API_VERSION=2023-10-01

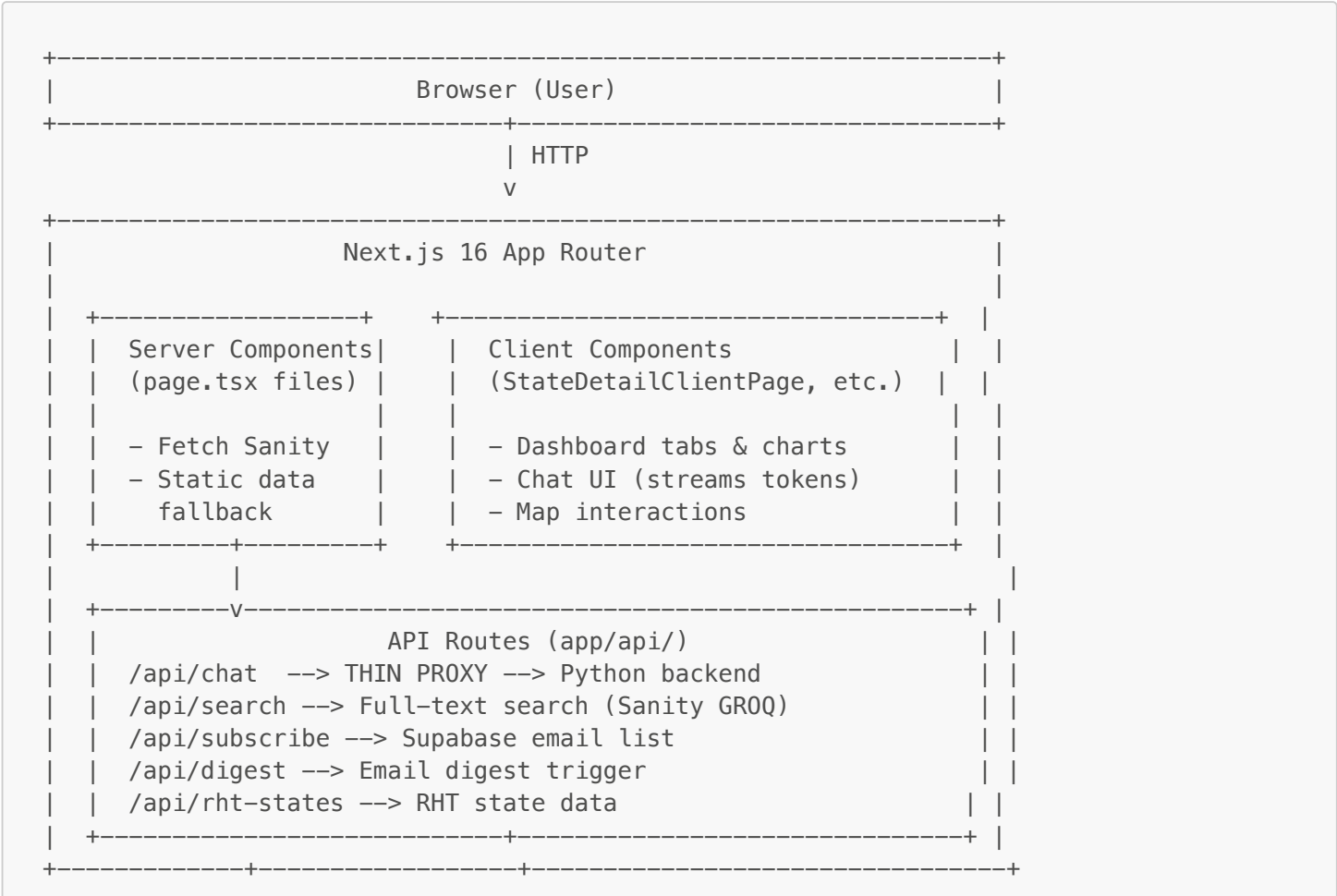
# ——— CORS ———
FRONTEND_URL=http://localhost:3000
```

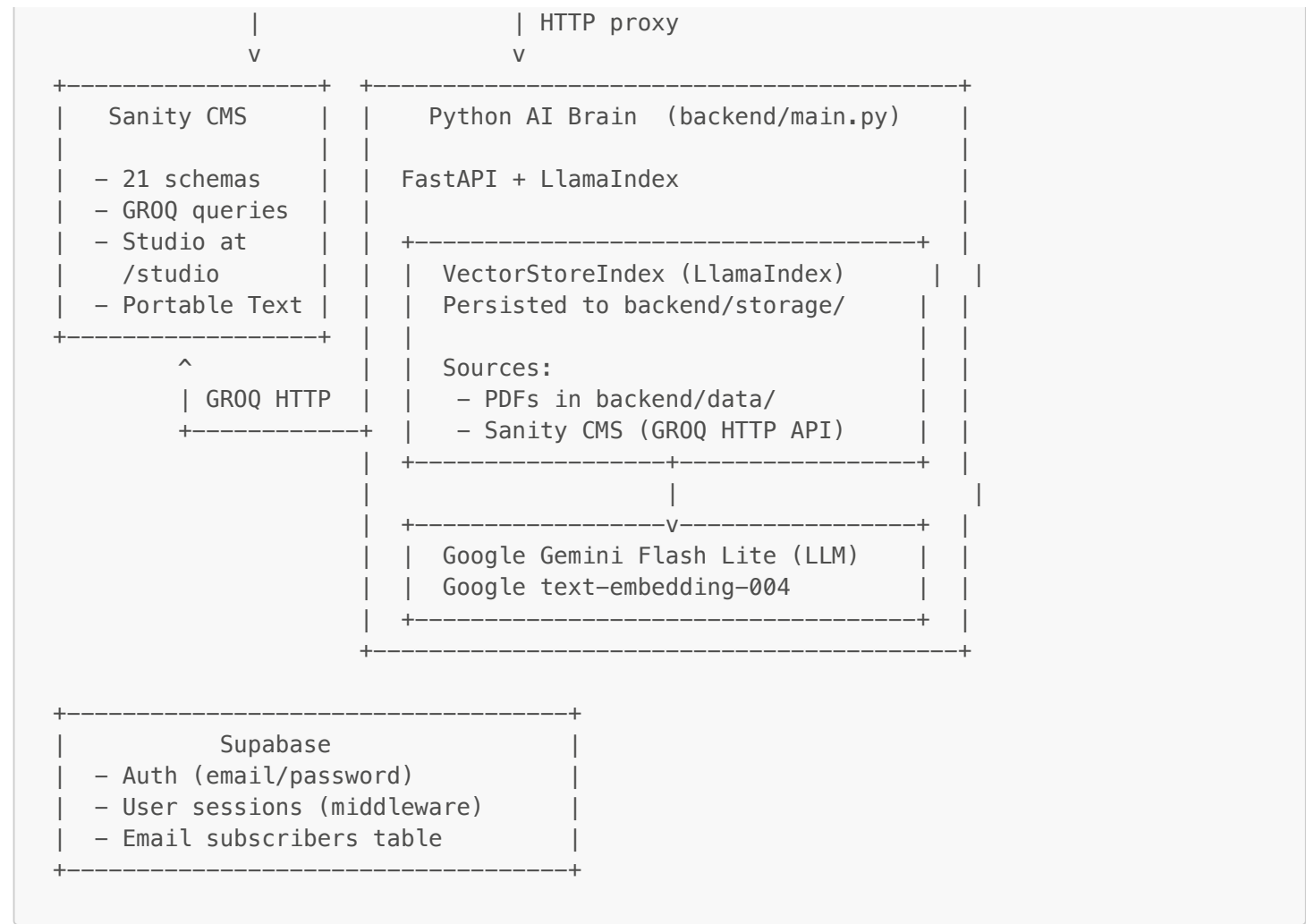
Variable reference table

Variable	File	Public?	Required	Used By
NEXT_PUBLIC_SANITY_PROJECT_ID	frontend/.env.local	Yes	Yes	Sanity client, seed scripts
NEXT_PUBLIC_SANITY_DATASET	frontend/.env.local	Yes	Yes	Sanity client
NEXT_PUBLIC_SANITY_API_VERSION	frontend/.env.local	Yes	Yes	Sanity client
SANITY_API_TOKEN	frontend/.env.local	No	Seeding only	Seed scripts
NEXT_PUBLIC_SUPABASE_URL	frontend/.env.local	Yes	Yes	Supabase client, middleware
NEXT_PUBLIC_SUPABASE_ANON_KEY	frontend/.env.local	Yes	Yes	Supabase browser client, auth
SUPABASE_SERVICE_ROLE_KEY	frontend/.env.local	No	Legacy only	sync-embeddings.ts
GEMINI_API_KEY	frontend/.env.local	No	Legacy only	sync-embeddings.ts
PYTHON_BACKEND_URL	frontend/.env.local	No	Yes	/api/chat/route.ts proxy
GOOGLE_API_KEY	backend/.env	No	Yes	LLM + embeddings in Python backend
SANITY_PROJECT_ID	backend/.env	No	Yes	Python Sanity ingestion
SANITY_DATASET	backend/.env	No	Yes	Python Sanity ingestion
SANITY_API_TOKEN	backend/.env	No	Yes	Python Sanity ingestion
SANITY_API_VERSION	backend/.env	No	No	Defaults to 2023-10-01
FRONTEND_URL	backend/.env	No	No	FastAPI CORS; defaults to http://localhost:3000

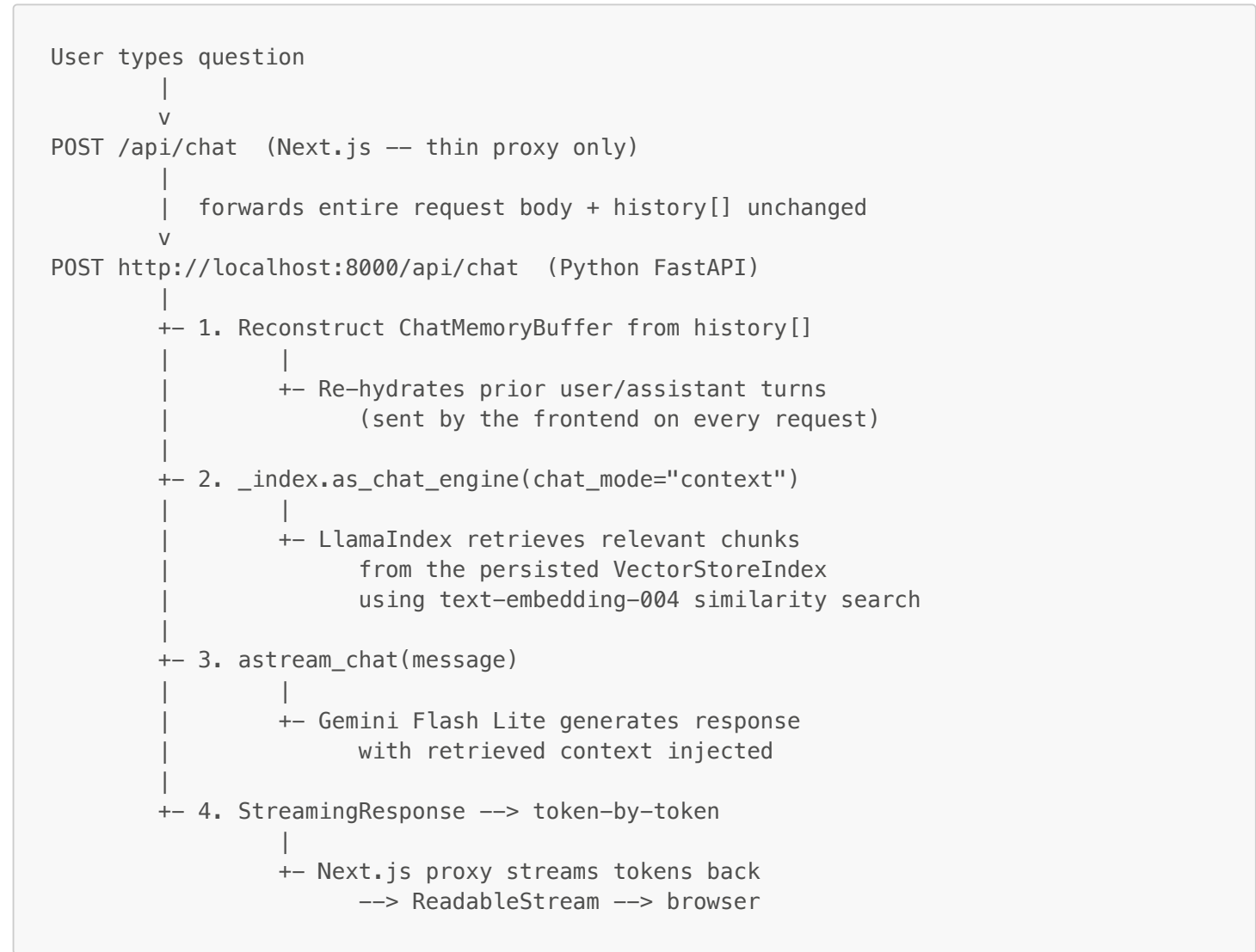
4. Architecture & Data Flow

High-level system diagram

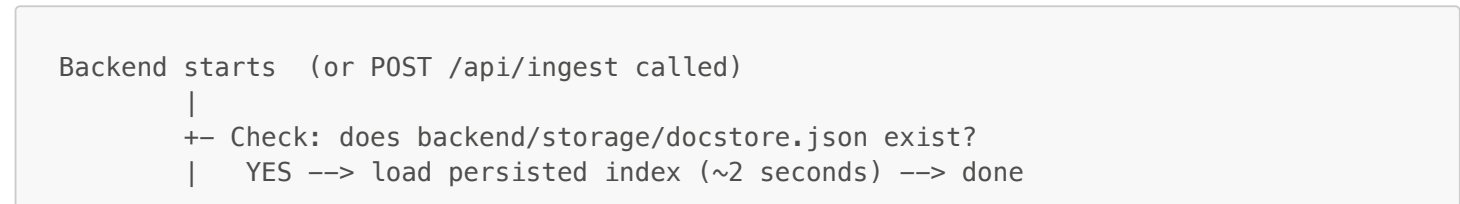




AI chat flow (detailed)



Index build flow (startup / on-demand)



```
| NO --> build from scratch:
|
+- 1. SimpleDirectoryReader(backend/data/)
|   --> loads all .pdf files as LlamaIndex Documents
|
+- 2. fetch_sanity_content()
|   --> GROQ HTTP API fetches 8 content types:
|       policyAnalysis, post, academyModule, caseStudy,
|       definition, analystNote, webinar, report
|   --> Flattens Portable Text:
|       string::join(body[].children[].text, " ")
|
+- 3. VectorStoreIndex.from_documents(all_docs)
|   --> Embeds every document with text-embedding-004
|   --> Builds in-memory similarity index
|
+- 4. index.storage_context.persist(backend/storage/)
|   --> Fast startup (~2s) on all subsequent runs
```

5. Getting Started (Local Dev)

Prerequisites

- **Node.js 20.x** + npm (for Next.js frontend)
- **Python 3.10+** + pip (for Python AI backend)
- Supabase project (Postgres + auth; pgvector optional — now used for auth only)
- Sanity project (free tier works)
- Google AI Studio API key (free tier: 60 req/min for Gemini, 1500 req/day for embeddings)

Terminal A — Python AI Backend (start first)

```
# 1. Set up Python environment
cd backend
python3 -m venv venv          # one time only
source venv/bin/activate     # macOS/Linux
# OR: venv\Scripts\activate  # Windows

# 2. Install dependencies
pip install -r requirements.txt

# 3. Create backend env file
cp .env.example .env
# Fill in: GOOGLE_API_KEY, SANITY_PROJECT_ID, SANITY_DATASET, SANITY_API_TOKEN

# 4. Start the AI backend
uvicorn main:app --reload --port 8000

# First run: builds the LlamaIndex from PDFs + Sanity (~2-5 min depending on content)
# Subsequent runs: loads persisted index from backend/storage/ (~2 seconds)
# Health check: http://localhost:8000/health
```

Adding PDFs: Drop any **.pdf** file into **backend/data/**, then call **POST /api/ingest** to rebuild the index with the new content. No code changes needed.

Terminal B — Next.js Frontend

```
# 1. Install dependencies
cd frontend
npm install

# 2. Create environment file
# Create frontend/.env.local with variables from $3
# Minimum required for local dev:
#   NEXT_PUBLIC_SANITY_PROJECT_ID, NEXT_PUBLIC_SANITY_DATASET
#   NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY
```

```
# PYTHON_BACKEND_URL=http://localhost:8000

# 3. Set up Supabase (one time only)
# Open Supabase dashboard → SQL Editor → run:
# supabase/setup-rag.sql

# 4. Start development server
npm run dev
# → http://localhost:3000
# Sanity Studio: http://localhost:3000/studio (embedded, no separate process)

# 5. (Optional) Seed initial content into Sanity
npx tsx scripts/seed-content.ts
npx tsx scripts/seed-hospitals.ts
npx tsx scripts/seed-sanity-rht.ts
npx tsx scripts/seed-sanity-performance-index.ts
```

After seeding Sanity content, call **POST** <http://localhost:8000/api/ingest> to rebuild the AI index with the new CMS content. The Python backend automatically re-ingests Sanity on every rebuild.

Build and lint

```
cd frontend
npm run build      # Production build
npm run start      # Production server
npm run lint       # ESLint
```

6. Routing & Page Structure

All routes use Next.js 14+ App Router with **page.tsx** server components as entry points.

Route map

URL Pattern	File	Type
/	app/page.tsx	Server
/policy	app/policy/page.tsx	Server → PillarHub template
/policy/[slug]	app/policy/[slug]/page.tsx	Server → ArticleEngine
/economics	app/economics/page.tsx	Server → PillarHub
/economics/[slug]	app/economics/[slug]/page.tsx	Server → ArticleEngine
/technology	app/technology/page.tsx	Server → PillarHub
/clinical	app/clinical/page.tsx	Server → PillarHub
/equity	app/equity/page.tsx	Server → PillarHub
/academy	app/academy/page.tsx	Server
/academy/modules/[slug]	app/academy/modules/[slug]/page.tsx	Server → AcademyModuleEngine
/academy/courses/[slug]	app/academy/courses/[slug]/page.tsx	Server
/academy/webinars/[slug]	app/academy/webinars/[slug]/page.tsx	Server
/academy/glossary	app/academy/glossary/page.tsx	Client (filter state)
/dashboard	app/dashboard/page.tsx	Server → DashboardIndexClient
/dashboard/[state]	app/dashboard/[state]/page.tsx	Server → StateDetailClientPage
/chat	app/chat/page.tsx	Client
/search	app/search/page.tsx	Client
/account	app/account/page.tsx	Client (protected)

URL Pattern	File	Type
/login	app/login/page.tsx	Client
/signup	app/signup/page.tsx	Client
/studio	Sanity Studio embedded	Studio

Naming conventions

- **Server components** (`page.tsx`) handle data fetching (Sanity GROQ queries, static data imports).
- **Client components** (`*Client*.tsx` or `"use client"` directive) handle interactivity — tab switches, map clicks, chat input.
- **Layout files** (`layout.tsx`) apply shared UI wrappers — the dashboard layout adds a sidebar, the root layout adds Header/AppShell/Footer.
- Dynamic segments use `[slug]` or `[state]` — state slugs are lowercase (e.g., `vermont`, `new-york`).

7. Component Architecture

Root layout chain

app/layout.tsx

- └─ CommandPalette ← Cmd+K global command palette
- TooltipProvider ← Hover tooltip context
- TickerProvider ← News ticker data context
- Header ← Top nav bar (links, auth state, search)
- AppShell ← Left sidebar + main content + right sidebar
 - HomeSidebar ← Left: pillar nav links
 - {children} ← Page content
 - RightSidebar ← Right: AI chat suggestions
- Footer

AppShell behavior

`AppShell` automatically collapses both sidebars on:

- Article routes (`/policy/[slug]`, `/economics/[slug]`, etc.)
- Academy module routes (`/academy/modules/[slug]`)

This gives article pages full-width layout. The sidebars show on hub pages, the home page, and the dashboard.

Template components (`components/templates/`)

Template	Used for	Key props
PillarHub	All 5 pillar landing pages	<code>pillarName</code> , <code>themeColor</code> , <code>featured</code> , <code>recent</code>
ArticleEngine	Policy analysis articles	<code>article</code> (full Sanity doc)
AcademyModuleEngine	Academy module content	<code>module</code> (Sanity academyModule doc)
AcademyModuleLayout	Module navigation wrapper	<code>prev</code> , <code>next</code> , <code>pillar</code>
HubPageTemplate	Generic hub pages (advisory)	<code>sections[]</code>

Dashboard components

- `app/dashboard/page.tsx` — Server component. Imports state list from `lib/data/states.ts`, renders `DashboardIndexClient`.
- `DashboardIndexClient.tsx` — Client. Shows US SVG map (`react-simple-maps`), handles state click → navigate to `/dashboard/[state]`.
- `app/dashboard/[state]/page.tsx` — Server. Looks up state in `rhtProgramData` and `performanceIndexData`, passes to `StateDetailClientPage`.
- `StateDetailClientPage.tsx` — Client. Three-tab UI: Performance Index | RHT Program | Hospital View.
- `RHTScorecard.tsx` — Renders the RHT metrics grid with color-coded status badges.
- `DashboardContext` (`lib/context/DashboardContext.tsx`) — Provides selected state, scenario mode (statusQuo | optimized), and all-states data to deeply nested dashboard components.

- **ProgramContext** (`lib/context/ProgramContext.tsx`) — Provides RHT program data to the RHT Program tab.

8. Color System

Single source of truth: `frontend/app/globals.css`

Do **not** add custom colors to `tailwind.config.js`. Tailwind v4 reads CSS `@theme` variables directly.

How it works

```
/* frontend/app/globals.css */
@theme {
  --color-brand-policy:      #0369a1; /* sky-700 */
  --color-brand-economics:  #059669; /* emerald-600 */
  --color-brand-technology: #4f46e5; /* indigo-600 */
  --color-brand-clinical:   #dc2626; /* red-600 */
  --color-brand-equity:     #d97706; /* amber-600 */
  --color-brand-academy:    #38bdf8; /* sky-400 */
  --color-brand-advisory:   #d946ef; /* fuchsia-500 */
  /* ... card and UI variables ... */
}
```

Tailwind v4 auto-generates utility classes from `@theme` variables:

- `text-brand-policy` → `color: #0369a1`
- `bg-brand-economics` → `background-color: #059669`
- `border-brand-clinical` → `border-color: #dc2626`

Complete color token reference

Token	Value	Usage
<code>--color-brand-policy</code>	<code>#0369a1</code>	Policy pillar (sky blue)
<code>--color-brand-economics</code>	<code>#059669</code>	Economics pillar (emerald)
<code>--color-brand-technology</code>	<code>#4f46e5</code>	Technology pillar (indigo)
<code>--color-brand-clinical</code>	<code>#dc2626</code>	Clinical pillar (red)
<code>--color-brand-equity</code>	<code>#d97706</code>	Equity pillar (amber)
<code>--color-brand-academy</code>	<code>#38bdf8</code>	Academy/education (light sky)
<code>--color-brand-advisory</code>	<code>#d946ef</code>	Advisory services (fuchsia)
<code>--color-card-policy</code>	(light sky bg)	Card backgrounds
<code>--color-card-tech</code>	(light indigo bg)	Card backgrounds
<code>--color-card-economics</code>	(light emerald bg)	Card backgrounds
<code>--color-card-clinical</code>	(light red bg)	Card backgrounds
<code>--color-card-equity</code>	(light amber bg)	Card backgrounds
<code>--color-ui-primary</code>	(brand accent)	Primary buttons/links
<code>--color-ui-border</code>	(slate)	Default borders
<code>--color-text-heading</code>	(near black)	H1–H6
<code>--color-text-body</code>	(slate-600)	Body text
<code>--color-surface</code>	white	Page background
<code>--color-surface-muted</code>	(slate-50)	Secondary backgrounds

Pillar theme objects

Components that need pillar-specific theming define a `styles` object (see `PillarHub.tsx` as the canonical example):

```
const styles = {
  policy:    { header: "bg-brand-policy", text: "text-brand-policy", ... },
  economics: { header: "bg-brand-economics", ... },
  technology: { header: "bg-brand-technology", ... },
  clinical:  { header: "bg-brand-clinical", ... },
  equity:    { header: "bg-brand-equity", ... },
};
const theme = styles[themeColor];
```

9. Sanity CMS — All Schemas

The studio lives at `/studio` (next-sanity embedded). Schema files are in `frontend/sanity/schemaTypes/`.

Document schemas

post — General Blog Articles

Field	Type	Notes
title	string	Required
slug	slug	Auto-generated from title
author	reference → author	
publishedAt	datetime	
mainImage	image	Cover image
categories	array of reference → category	
body	blockContent	Full rich text

policyAnalysis — Policy Deep-Dives

Field	Type	Notes
title	string	Required
slug	slug	Required
pillar	string	Policy Economics Technology Clinical Equity
summary	text	Executive summary
impactLevel	string	Critical High Medium Low
category	string	Thematic category
publishedAt	datetime	
body	blockContent	Full analysis

Consumed by: Pillar hub pages (featured + recent), ArticleEngine template, RAG sync.

caseStudy — Case Studies

Field	Type	Notes
title	string	Required
slug	slug	Required
pillar	string	Policy Economics Technology Clinical Equity
clientType	string	e.g., "Rural Hospital", "Payer"
summary	text	Executive summary
metrics	array of string	e.g., "40% Reduction"
body	blockContent	Full case study

Field	Type	Notes
mainImage	image	Cover image

course — Academy Courses

Field	Type	Notes
title	string	Required
slug	slug	Required
pillar	string	Policy Economics Technology Clinical Equity
type	string	CERTIFICATION COURSE WEBINAR MASTERCLASS
description	text	Short description
meta	string	e.g., "8 Weeks • Online Cohort"
price	string	e.g., "\$2,995"
instructors	array of reference → instructor	
modules	array of reference → academyModule	Ordered module list
overview	blockContent	Full syllabus

academyModule — Academy Modules (individual lessons)

Field	Type	Notes
title	string	Required
slug	slug	Required
pillar	string	Policy Economics Technology Clinical Equity
moduleNumber	number	Order within course
summary	text	Short description
learningObjectives	array of string	Bullet objectives
prevModule	reference → academyModule	Navigation
nextModule	reference → academyModule	Navigation
body	blockContent	Full module content

webinar — Webinars

Field	Type	Notes
title	string	Required
slug	slug	Required
pillar	string	Policy Economics Technology Clinical Equity
description	text	
youtubeId	string	YouTube video ID
scheduledDate	datetime	
speakers	array of reference → instructor	

report — Research Reports

Field	Type	Notes
title	string	Required
slug	slug	Required

Field	Type	Notes
<code>pillar</code>	string	Policy Economics Technology Clinical Equity
<code>abstract</code>	text	Executive summary
<code>publishedAt</code>	datetime	
<code>body</code>	blockContent	Full report
<code>pdfUrl</code>	url	Download link

analystNote — Analyst Notes (for RAG context)

Field	Type	Notes
<code>title</code>	string	
<code>pillar</code>	string	Pillar association
<code>body</code>	blockContent	Note content

These are internal notes embedded into the RAG context but not displayed publicly.

definition — Glossary Terms

Field	Type	Notes
<code>term</code>	string	Required — term or acronym
<code>description</code>	text	Required — full definition
<code>pillars</code>	array of string	Tags: Policy Economics Technology Clinical Equity

Consumed by: `/academy/glossary` page (filterable by pillar), RAG sync.

instructor — Academy Instructors

Field	Type	Notes
<code>name</code>	string	
<code>bio</code>	text	
<code>photo</code>	image	
<code>title</code>	string	Professional title
<code>organization</code>	string	Affiliation

author — Article Authors

Field	Type	Notes
<code>name</code>	string	
<code>bio</code>	text	
<code>image</code>	image	

category — Content Categories

Field	Type	Notes
<code>title</code>	string	
<code>description</code>	text	

hospital — Hospital Records

Field	Type	Notes
-------	------	-------

Field	Type	Notes
name	string	
stateSlug	string	Lowercase state slug (e.g., "vermont")
location	string	City, state
type	string	Critical Access Regional Academic Community
beds	number	Licensed bed count
margin	number	Operating margin %
rhtParticipant	boolean	

rhtState — RHT Program State Profiles (Sanity version)

Mirrors the structure of `lib/data/rht-program.ts` but managed in Sanity.

Field	Type	Notes
stateId	string	Lowercase slug (e.g., "vermont")
stateName	string	Display name
awardAmount	string	Dollar amount
status	string	Active Pending At Risk
description	text	Program overview
initiatives	array of objects	<code>{ title, description }</code>
metrics	array of objects	<code>{ label, status, target }</code>

statePerformanceIndex — State Performance (Sanity version)

Sanity-managed version of `lib/data/performance-index-data.ts`.

subscriber — Email Subscribers

Field	Type	Notes
email	string	Required
subscribedAt	datetime	
active	boolean	

ticker — News Ticker Items

Field	Type	Notes
label	string	Short ticker text
url	url	Link target
publishedAt	datetime	

dailyInsight — Daily Insights

Field	Type	Notes
title	string	
body	blockContent	
publishedAt	datetime	
pillar	string	

audio — Audio Content (Podcasts)

Field	Type	Notes
title	string	
description	text	
audioUrl	url	Direct audio file URL
pillar	string	
publishedAt	datetime	

blockContent — Shared Rich Text Type

Not a document — a reusable type for portable text fields across all schemas. Supports: paragraphs, headings, blockquotes, images, code blocks, and custom inline annotations.

10. Static Data Layer

Static TypeScript data files in `frontend/lib/data/` power the dashboard while Sanity-managed versions are being built out. They are the source of truth for the dashboard currently.

performance-index-data.ts

Type:

```
type PerformanceIndexProfile = {
  id: string; // lowercase state slug (e.g., "vermont")
  stateName: string;
  overallScore: number; // 0–100 composite
  rank: number; // 1–50 national rank
  tier: "Leader" | "Advancing" | "Developing" | "Lagging";
  lastUpdated: string;
  metrics: {
    policy: {
      vbpAdoption: number; // 0–100 score
      telehealth: number;
      scopeOfPractice: number;
    };
    economics: {
      spendingPerCapita: number;
      workforceGaps: number;
      insuranceCoverage: number;
    };
    technology: {
      hieAdoption: number;
      broadbandAccess: number;
      ehrAdoption: number;
    };
    clinical: {
      preventiveCare: number;
      readmissionRate: number;
      chronicDiseaseControl: number;
    };
    equity: {
      racialEquityGap: number;
      ruralUrbanGap: number;
      sdohIntegration: number;
    };
  };
};
```

All 50 states have entries. Scores are on a 0–100 scale (higher = better). The file was updated (2026-03-10) to include all 5 pillar metric groups — earlier it only had policy, economics, and technology.

rht-program.ts

```
type RHTProfile = {
  id: string; // "vermont"
  stateName: string; // "Vermont"
  awardAmount: string; // "$195,000,000"
  status?: "Active" | "Pending" | "At Risk";
  strategicFocus: string | string[];
  description: string; // Program overview paragraph
  initiatives: {
    title: string;
    description: string;
  }[];
  metrics: {
    label: string;
    status: "Pending" | "In Progress" | "Achieved";
    target?: string;
  }[];
};

export const rhtProgramData: Record<string, RHTProfile>
```

Only states with active CMS Innovation RHT awards have entries.

hospital-data.ts

Static hospital reference data. Each entry includes name, stateSlug, location, type, beds, margin, and rhtParticipant flag. This data is shown in the "Hospital View" tab on state detail pages.

states.ts

Lookup table mapping state slugs to display names and two-letter abbreviations. Used by the dashboard map and state selector.

rht-awards.ts

Award amounts by state slug. Cross-references [rht-program.ts](#).

11. RAG Pipeline (AI Analyst)

The AI Analyst uses a **Python-first architecture** — all RAG, embedding, LLM inference, and conversation memory live in the Python backend ([backend/main.py](#)). The Next.js layer is a thin HTTP proxy with no AI logic.

Components

File	Role
backend/main.py	FastAPI server: ingestion, VectorStoreIndex, chat endpoint, streaming
backend/data/*.pdf	Source PDFs ingested at startup (drop files here + call /api/ingest)
backend/storage/	Persisted LlamaIndex VectorStoreIndex — gitignored, auto-generated
backend/.env	Backend secrets: GOOGLE_API_KEY, Sanity vars
frontend/app/api/chat/route.ts	Thin proxy only — forwards request, streams response back

Legacy files (kept for reference, no longer used for chat):

- [frontend/lib/rag.ts](#) — TypeScript RAG helpers (Supabase pgvector approach)
- [frontend/scripts/sync-embeddings.ts](#) — TypeScript Sanity→Supabase embedding sync

Python backend ([backend/main.py](#))

Startup behaviour

```
# Startup checks for a persisted index first:
if os.path.exists("backend/storage/docstore.json"):
    # FAST PATH: load in ~2 seconds
    storage_context = StorageContext.from_defaults(persist_dir=STORAGE_DIR)
```

```
    _index = load_index_from_storage(storage_context)
else:
    # SLOW PATH: build from scratch (~2-5 min first run)
    _index = await build_index()
```

Content sources ingested

On build, the index combines:

- 1. **PDFs** — All `.pdf` files in `backend/data/` via `SimpleDirectoryReader`
- 2. **Sanity CMS** — 8 content types fetched via GROQ HTTP API:

Sanity Type	GROQ projection
<code>policyAnalysis</code>	title, pillar, summary, body (flattened)
<code>post</code>	title, body (flattened)
<code>academyModule</code>	title, pillar, summary, learningObjectives, body
<code>caseStudy</code>	title, pillar, summary, body
<code>definition</code>	term, description, pillars
<code>analystNote</code>	title, pillar, body
<code>webinar</code>	title, pillar, description
<code>report</code>	title, pillar, abstract

Portable Text is flattened using the GROQ expression: `string::join(body[].children[].text, " ")`

Each document is truncated to 8,000 characters before embedding.

Chat endpoint internals

```
@app.post("/api/chat")
async def chat(request: ChatRequest):
    # 1. Rebuild conversation memory from history sent by the frontend
    memory = ChatMemoryBuffer.from_defaults(token_limit=4096)
    for msg in request.history:
        if msg["role"] == "user":
            memory.put(ChatMessage(role=MessageRole.USER, content=msg["text"]))
        elif msg["role"] == "ai":
            memory.put(ChatMessage(role=MessageRole.ASSISTANT, content=msg["text"]))

    # 2. Create context-aware chat engine using the persisted index
    chat_engine = _index.as_chat_engine(
        chat_mode="context", # retrieves relevant chunks per query
        memory=memory,
        system_prompt=HTR_ANALYST_PERSONA,
        verbose=False,
    )

    # 3. Stream response token-by-token
    async def generate():
        streaming_response = await chat_engine.astream_chat(request.message)
        async for token in streaming_response.async_response_gen():
            yield token

    return StreamingResponse(generate(), media_type="text/plain; charset=utf-8")
```

Available endpoints

Method	Endpoint	Description
POST	<code>/api/chat</code>	RAG-enhanced streaming chat. Body: { <code>message</code> , <code>history[]</code> , <code>temperature?</code> , <code>systemPrompt?</code> }

Method	Endpoint	Description
POST	/api/ingest	Trigger async index rebuild (background task). Call after adding PDFs or publishing Sanity content.
GET	/health	Returns index status, model names, whether index is persisted.

Next.js proxy (`frontend/app/api/chat/route.ts`)

```
const PYTHON_BACKEND = process.env.PYTHON_BACKEND_URL || "http://localhost:8000";

export async function POST(req: Request) {
  const body = await req.json();
  const upstream = await fetch(`${PYTHON_BACKEND}/api/chat`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(body),
  });
  // Stream the response directly back to the browser
  return new NextResponse(upstream.body, {
    headers: { "Content-Type": "text/plain; charset=utf-8" },
  });
}
```

The frontend sends `{ message, history[], temperature, systemPrompt }` — the Python backend uses `history[]` to reconstruct the `ChatMemoryBuffer` for each request.

Adding PDFs to the knowledge base

1. Drop any `.pdf` file into `backend/data/`
2. Call `POST http://localhost:8000/api/ingest` (or restart the server)
3. The backend rebuilds the index in the background — logs show progress
4. No code changes or restarts of the Next.js app needed

Tuning parameters

Parameter	Location	Current	Effect
Model	backend/main.py	gemini-flash-lite-latest	Chat quality vs. latency
Embed model	backend/main.py	text-embedding-004	768-dim, fixed
Memory limit	backend/main.py	token_limit=4096	Longer context = more history retained
Text truncation	backend/main.py	8,000 chars per doc	Larger = richer context, slower embed
Temperature	Request body	0.7	Higher = more creative, less reliable

12. Dashboard System

State detail pages

URL pattern: `/dashboard/[state]` where `state` is a lowercase slug (e.g., `vermont`, `new-york`).

Server component (`app/dashboard/[state]/page.tsx`):

- Queries Sanity first via `getPerformanceIndex()` and `getRhtState()` (`lib/sanity-dashboard-queries.ts`)
- If Sanity has no document for this state yet, falls back to static data files (`lib/data/performance-index-data.ts`, `lib/data/rht-program.ts`)
- If neither source has data → returns 404
- Passes resolved data to `StateDetailClientPage`

```
const indexData = sanityIndex ?? performanceIndexData[stateSlug] ?? null;
const programData = sanityProgram ?? rhtProgramData[stateSlug] ?? null;
if (!indexData && !programData) return notFound();
```

Client component (`StateDetailClientPage.tsx`):

- Three tabs: **Performance Index** | **RHT Program** | **Hospital View**
- Tab state managed with `useState`

Performance Index tab

Renders 5 pillar metric groups:

1. **Policy** — VBP Adoption, Telehealth Coverage, Scope of Practice
2. **Economics** — Spending Per Capita, Workforce Gaps, Insurance Coverage
3. **Technology** — HIE Adoption, Broadband Access, EHR Adoption
4. **Clinical** — Preventive Care, Readmission Rate, Chronic Disease Control
5. **Equity** — Racial Equity Gap, Rural-Urban Gap, SDOH Integration

Each metric renders with a `MetricDisplay` component showing score, color-coded bar, and label.

Hospital solvency simulation

The "Hospital View" tab uses the `useSolvencySimulation` hook (`lib/hooks/useSolvencySimulation.ts`) to model two scenarios:

- **Status Quo** — current hospital financials
- **Optimized** — projected financials with RHT program interventions

Controlled via `DashboardContext.simulationMode`.

Contexts

DashboardContext (provides to all dashboard subtree):

```
{
  selectedStateId: string | null;
  setSelectedStateId: (id) => void;
  simulationMode: 'statusQuo' | 'optimized';
  setSimulationMode: (mode) => void;
  allStates: Record<string, RHTProfile>;
  selectedStateData: RHTProfile | null;
}
```

ProgramContext — Provides RHT program data (initiatives, metrics) to the RHT Program tab and scorecard.

13. Auth & Middleware

Supabase Auth

Authentication uses Supabase email/password auth. The Supabase client is initialized in two ways:

1. **Browser client** (`lib/supabase.ts`) — uses anon key, for client-side session reads:

```
export const supabase = createClient(SUPABASE_URL, SUPABASE_ANON_KEY)
```

2. **Server client** (middleware, `@supabase/ssr`) — uses `createServerClient` with cookie handling for SSR session validation.

Middleware (`middleware.ts`)

Runs on edge runtime before every matched request. Checks Supabase session:

```
// Only runs for:
export const config = { matcher: ["/account/:path*"] };

// Logic:
// 1. If path is not protected → NextResponse.next()
```

```
// 2. Create server-side Supabase client with request cookies
// 3. supabase.auth.getUser() – validates session
// 4. If no user → redirect to /login?from={pathname}
// 5. If authenticated → NextResponse.next() with updated cookies
```

Auth pages

- `/login` — Email + password sign-in form
- `/signup` — New account creation
- `/reset-password` — Password reset request and confirm flow
- `/account` — Protected. Shows user profile, subscription status.

Extending auth

To protect additional routes, add the path prefix to `PROTECTED_PREFIXES` in `middleware.ts`:

```
const PROTECTED_PREFIXES = ["/account", "/your-new-protected-route"];
```

14. Scripts Reference

All scripts run from the `frontend/` directory using `npx tsx`.

Execution order for a fresh setup

```
# 1. DB setup (Supabase SQL Editor – not a script)
# Run: supabase/setup-rag.sql

# 2. Seed Sanity content (run in order)
npx tsx scripts/seed-content.ts           # Articles, policy analyses
npx tsx scripts/seed-hospitals.ts        # Hospital records
npx tsx scripts/seed-sanity-rht.ts       # RHT state profiles
npx tsx scripts/seed-sanity-performance-index.ts # State performance data

# 3. Sync RAG embeddings (after Sanity has content)
npx tsx scripts/sync-embeddings.ts
```

Script details

`seed-content.ts`

Seeds initial policyAnalysis and post documents into Sanity. Run once per fresh project setup or to reset demo content. Uses `SANITY_API_TOKEN` for write access.

`seed-hospitals.ts`

Seeds hospital documents into Sanity from a static data source. Matches state slugs from `lib/data/states.ts`.

`seed-sanity-rht.ts`

Converts `lib/data/rht-program.ts` entries into Sanity `rhtState` documents. Keeps the Sanity CMS in sync with the static data.

`seed-sanity-performance-index.ts`

Converts `lib/data/performance-index-data.ts` entries into Sanity `statePerformanceIndex` documents.

`sync-embeddings.ts`

The most important operational script. Run after any significant Sanity content updates:

- Fetches all embeddable content types from Sanity
- Generates 768-dim embeddings via Google text-embedding-004

- Upserts into Supabase `content_embeddings` table
- Handles rate limiting (1s sleep between API calls)
- Reports: `N synced`, `N skipped`, `N errors`

Required env vars for `sync-embeddings.ts`:

- `GEMINI_API_KEY`
- `SANITY_API_TOKEN`
- `NEXT_PUBLIC_SANITY_PROJECT_ID`
- `NEXT_PUBLIC_SUPABASE_URL`
- `SUPABASE_SERVICE_ROLE_KEY`

15. API Routes

POST /api/chat

Thin proxy to the Python AI backend. Forwards the full request body and streams the response back. See §11 for full details.

Body: `{ message: string, history?: {role, text}[], temperature?: number, systemPrompt?: string }` **Response:** Streaming text (ReadableStream) — tokens from Gemini Flash Lite via LlamaIndex **Auth:** None required (public endpoint) **Requires:** Python backend running at `PYTHON_BACKEND_URL` (default: `http://localhost:8000`)

GET /api/rht-states

Returns RHT program state profiles as JSON.

Response: `{ states: Record<string, RHTProfile> }` **Source:** `lib/data/rht-program.ts`

POST /api/subscribe

Adds an email subscriber to Supabase.

Body: `{ email: string }` **Response:** `{ success: boolean }` **Storage:** Supabase `subscriber` table (or Sanity `subscriber` document)

POST /api/digest

Triggers generation and delivery of the weekly email digest.

Body: `{ to?: string }` (optional override recipient) **Response:** `{ sent: boolean }`

GET /api/search

Full-text content search.

Query params: `?q=search+term` **Response:** `{ results: SearchResult[] }` **Source:** Sanity GROQ query with text matching

16. How to Add a New Pillar

Adding a new pillar requires changes in 12+ places. Follow this checklist:

1. Globals CSS — add color tokens

```
/* frontend/app/globals.css */
@theme {
  --color-brand-newpillar: #your-color;
  --color-card-newpillar: #your-light-color;
}
```

2. Create the pillar route directory

```
frontend/app/newpillar/
├─ page.tsx           ← Server component using PillarHub template
```

└─ [slug]/

└─ page.tsx

← Article page using ArticleEngine template

3. **PillarHub.tsx** — extend the types and styles

```
// Update the union type:
themeColor: "economics" | "policy" | "technology" | "clinical" | "equity" | "newpillar"

// Add to the styles object:
newpillar: {
  header: "bg-brand-newpillar",
  text: "text-brand-newpillar",
  border: "border-brand-newpillar",
  indicator: "bg-brand-newpillar",
  hoverText: "group-hover:text-brand-newpillar",
  lightBg: "bg-[your-light-color]-50",
},
```

4. All Sanity schemas — add to pillar lists

Update the **list** option in every schema that has a **pillar** field:

- `sanity/schemaTypes/policyAnalysis.ts`
- `sanity/schemaTypes/caseStudy.ts`
- `sanity/schemaTypes/course.ts`
- `sanity/schemaTypes/academyModule.ts`
- `sanity/schemaTypes/webinar.ts`
- `sanity/schemaTypes/report.ts`
- `sanity/schemaTypes/analystNote.ts`

```
// In each schema's pillar field:
options: {
  list: ['Policy', 'Economics', 'Technology', 'Clinical', 'Equity', 'New Pillar'],
}
```

5. **definition.ts** (Sanity schema) — add to pillars array

```
options: {
  list: [
    ...existing entries...,
    { title: 'New Pillar', value: 'New Pillar' },
  ],
}
```

6. **VideoLibrary.tsx** — add to `getPillarTheme()` switch and `pillarOrder`

```
case "New Pillar":
  return { text: "text-brand-newpillar", border: "border-brand-newpillar", ... };

const pillarOrder = ['Policy', 'Economics', 'Technology', 'Clinical', 'Equity', 'New Pillar', 'General']
```

7. Glossary page — add pillar to list and badge/button styles

`frontend/app/academy/glossary/page.tsx`:

- Add to **pillars** array
- Add case to `getBadgeStyle()`
- Add case to `getPillarButtonStyle()`

8. `performance-index-data.ts` — add new metrics to all 50 states

```
// Add to PerformanceIndexProfile type:
newpillar: {
  metric1: number;
  metric2: number;
  metric3: number;
};

// Add to all 50 state entries:
newpillar: { metric1: 75, metric2: 80, metric3: 65 },
```

9. `StateDetailClientPage.tsx` — add metric section to Performance Index tab

Import an icon, then add a new `<div>` block mirroring the existing clinical/equity sections.

10. Navigation — update Header, HomeSidebar, and Footer

- `components/Header.tsx` — add pillar to the nav dropdown
- `components/HomeSidebar.tsx` — add direct link
- `components/Footer.tsx` — add to pillars list

11. Re-run sync after adding Sanity content

```
npx tsx scripts/sync-embeddings.ts
```

17. How to Add a New State

States appear in the dashboard, the US map, and RHT program profiles.

Step 1: Add to `lib/data/states.ts`

```
{ id: "new-state", name: "New State", abbreviation: "NS" }
```

Step 2: Add to `lib/data/performance-index-data.ts`

Add a full `PerformanceIndexProfile` entry:

```
"new-state": {
  id: "new-state",
  stateName: "New State",
  overallScore: 72,
  rank: 26,
  tier: "Advancing",
  lastUpdated: "2026-01-01",
  metrics: {
    policy: { vbpAdoption: 70, telehealth: 65, scopeOfPractice: 75 },
    economics: { spendingPerCapita: 80, workforceGaps: 60, insuranceCoverage: 85 },
    technology: { hieAdoption: 70, broadbandAccess: 55, ehrAdoption: 78 },
    clinical: { preventiveCare: 72, readmissionRate: 68, chronicDiseaseControl: 74 },
    equity: { racialEquityGap: 60, ruralUrbanGap: 55, sdohIntegration: 65 },
  },
},
```

Step 3: (Optional) Add RHT profile to `lib/data/rht-program.ts`

Only if the state has an active RHT award:

```
"new-state": {
  id: "new-state",
```

```
stateName: "New State",
awardAmount: "$50,000,000",
status: "Active",
strategicFocus: "...",
description: "...",
initiatives: [...],
metrics: [...],
},
```

Step 4: Add hospital data to `lib/data/hospital-data.ts`

Add relevant hospitals with `stateSlug: "new-state"`.

Step 5: Seed to Sanity (optional but recommended)

```
# The seed scripts will pick up the new state data:
npx tsx scripts/seed-sanity-rht.ts
npx tsx scripts/seed-sanity-performance-index.ts
```

Step 6: Verify the map

The US map in `DashboardIndexClient.tsx` uses `react-simple-maps` with GeoJSON — all 50 US states are already present in the TopoJSON. No map changes needed. The state becomes clickable automatically once its slug matches a known state name in the data.

18. Deployment

The system has **two deployable services**: the Next.js frontend and the Python AI backend. Deploy them independently.

Frontend — Vercel (recommended)

```
cd frontend
npx vercel --prod
```

No `vercel.json` required — defaults handle everything.

Set all frontend env vars in Vercel → Settings → Environment Variables. Mark `SANITY_API_TOKEN`, `SUPABASE_SERVICE_ROLE_KEY` as server-only. Set `PYTHON_BACKEND_URL` to your production backend URL.

Python Backend — options

The Python backend is a long-running async process. Recommended hosts:

Host	Notes
Railway	Easiest — connect repo, set env vars, auto-deploys on push
Render	Free tier available; set start command: <code>uvicorn main:app --host 0.0.0.0 --port \$PORT</code>
Fly.io	Good for low-latency; Docker-based
Local (dev)	<code>uvicorn main:app --reload --port 8000</code>

Important for production backend:

- The `backend/storage/` directory (persisted index) is gitignored. On first deployment the backend will build the index from scratch — ensure `GOOGLE_API_KEY` and Sanity vars are set before first startup.
- To persist the index across deploys, mount a persistent volume at `backend/storage/` (Railway and Fly.io both support this). Without a volume, every redeploy rebuilds from scratch.
- Set `FRONTEND_URL` to your production Next.js domain for correct CORS headers.

Pre-deployment checklist

- ☐ `frontend/.env.local` vars set in Vercel environment

- ☐ `PYTHON_BACKEND_URL` set to production backend URL in Vercel
- ☐ `backend/.env` vars set in backend host environment
- ☐ `supabase/setup-rag.sql` run against production Supabase project
- ☐ Sanity dataset is `production` (not a dev dataset)
- ☐ `npm run build` completes without errors locally
- ☐ Python backend health check passes: `GET /health` returns `{ index_ready: true }`
- ☐ No hardcoded `localhost` URLs in components

Build notes

```
cd frontend
npm run build      # Must pass before deploying
npm run start      # Test production build locally
```

Tailwind v4 note: The beta PostCSS plugin ([@tailwindcss/postcss](#)) handles CSS compilation at build time. If build fails with CSS errors, ensure `postcss.config.js` contains:

```
module.exports = { plugins: [ '@tailwindcss/postcss' ] }
```

Database migrations

No migration files — Supabase schema changes applied manually via the SQL editor. Before any production DB change:

1. Test in a staging Supabase project
2. Apply to production

Sanity Studio deployment

Embedded in the Next.js app at `/studio`. Deploys automatically with the Next.js app. Configure CORS:

- [sanity.io/manage](#) → your project → API → CORS Origins
- Add: `https://your-production-domain.com`

19. Known Issues & Roadmap

Known issues

Issue	Status	Notes
Python backend not auto-started	Active	Must be started manually alongside <code>npm run dev</code> . No process manager yet.
<code>backend/storage/</code> not persisted on cloud deploys	Active	Without a mounted volume, every redeploy rebuilds the index from scratch (~2-5 min downtime).
TypeScript <code>any</code> types in Sanity queries	Active	GROQ results typed as <code>any</code> — <code>sanity typegen</code> not run
<code>react-simple-maps</code> SSR warning	Active	Map component requires client-side rendering guard
Dashboard static data fallback	Partial	Sanity → static file fallback is wired. Static files are source of truth until Sanity seed scripts are run.
Content generator uses Google SDK directly	Active	<code>generate_sanity_content.py</code> uses <code>google-generativeai</code> — add to Python venv or run separately

Resolved (this session)

Issue	Resolution
Academy filter buttons decorative-only	Extracted to <code>CoursesClient.tsx</code> with <code>useState</code> — filters are now functional
<code>TypeError on /dashboard/[state]</code> (<code>preventiveCare</code> undefined)	Added <code>clinical/equity</code> to GROQ projection with <code>coalesce(..., 0)</code> fallbacks; added fields to Sanity schema

Issue	Resolution
AI chat had no conversation memory	Frontend sends <code>history[]</code> on every request; Python backend reconstructs <code>ChatMemoryBuffer</code>
AI logic entirely in TypeScript	Python backend (FastAPI + LlamaIndex) now owns all RAG; Next.js is a thin proxy
Pillar hub cards no hover background	Added <code>hover:bg-{color}-50/80</code> to all pillar page cards and PillarHub template

Roadmap (prioritized)

- Python backend process management** — Add a `Procfile` or `docker-compose.yml` so `make dev` starts both Next.js and uvicorn with one command.
- Persistent index storage** — Mount a volume at `backend/storage/` in production (Railway/Fly.io) so the index survives redeloys.
- Automated RAG refresh** — Sanity webhook → call `POST /api/ingest` on content publish. No manual trigger needed.
- Search** — Wire up `/api/search` with Sanity's native search or Algolia. Route file exists; needs GROQ expansion.
- Live data pipeline** — Replace remaining `lib/data/*.ts` static files with Sanity GROQ queries for dashboard data (partially done — fallback wired, Sanity schemas exist).
- TypeScript strictness** — `npx sanity typegen generate` for typed GROQ results; eliminate `any` in routes.
- Email digest** — Complete `/api/digest` endpoint to generate weekly summary from Sanity and send to subscriber list.
- Auth + personalization** — Saved states, reading history, personalized recommendations via Supabase RLS.
- Content depth** — Populate all 5 pillar hubs; add Clinical and Equity academy modules; more PDFs to `backend/data/`.
- Performance** — Sanity CDN (`useCdn: true`) for production reads; ISR for static-heavy pages.

Last updated: 2026-03-10 (Python AI backend integration)